23

完整的虚拟内存系统

在结束对内存虚拟化（memory virtualization）的学习之前，我们来更仔细地看看完整的虚拟内存（virtual memory）系统是怎样搭建起来的。我们已经见过这类系统的关键要素，包括各种页表（page table）设计、与 TLB 的交互（有时甚至由操作系统自己处理），以及决定哪些页（page）留在内存、哪些页踢出内存的策略。然而，一个完整的虚拟内存系统还包含许多其他特性，涉及性能、功能和安全等诸多方面。于是，我们的关键问题是：

关键问题：如何构建完整的虚拟内存系统
实现一个完整的虚拟内存系统需要哪些特性？它们如何提高性能、增强安全性，或以其他方式改进系统？

我们会通过介绍两个系统来回答这个问题。第一个是最早的“现代”虚拟内存管理器之一，即 **VAX/VMS** 操作系统中的虚拟内存管理器 [LL82]，它开发于 20 世纪 70 年代到 80 年代初；这个系统里有多得惊人的技术和方法一直沿用至今，因此非常值得研究。有些思想即使已有 50 年历史，仍然值得了解；这个道理在其他大多数领域（比如物理学）的人看来再平常不过，但在技术驱动的学科（比如计算机科学）里却得专门说一说。

第二个是 **Linux**，原因应该不言而喻。Linux 应用广泛，能高效地运行在各种系统上，小到性能孱弱的手机，大到现代数据中心里可扩展性（scalability）最强的多核（multicore）系统。因此，它的虚拟内存系统必须足够灵活，才能在所有这些场景下都顺利运行。我们会逐一讨论这两个系统，看看前几章介绍的概念是如何在一个完整的内存管理器中融为一体的。

23.1 VAX/VMS 虚拟内存

VAX-11 小型机（minicomputer）体系结构由**数字设备公司**（Digital Equipment Corporation，**DEC**）于 20 世纪 70 年代末推出。在小型机时代，DEC 是计算机行业的巨头；可惜，一系列错误的决策，再加上 PC 的兴起，让它慢慢地（但不可避免地）走向了消亡 [C03]。这个体系结构有好几种实现，包括 VAX-11/780 和性能稍弱的 VAX-11/750。

这个系统的操作系统叫 VAX/VMS（或者干脆叫 VMS），它的主要设计者之一是 Dave Cutler，他后来主持开发了微软的 Windows NT [C93]。VMS 面临一个通用性问题：它要运行在各种各样的机器上，从非常便宜的 VAXen（没错，复数就是这么写的），到同一体系

结构家族中极其高端、强大的机器。因此，操作系统的机制和策略必须在差异如此之大的各种系统上都能工作（而且要工作得很好）。

另外，VMS 还是一个很好的例子，展示了如何用软件创新来掩盖体系结构的一些固有缺陷。操作系统常常要依靠硬件来构建高效的抽象（abstraction）和假象，但硬件设计者有时并不能把每件事都做对；我们会在 VAX 硬件中看到几个这样的例子，也会看到 VMS 操作系统如何在硬件存在这些缺陷的情况下，依然构建出一个有效、能用的系统。

内存管理硬件

VAX-11 为每个进程（process）提供了一个 32 位的虚拟地址空间（virtual address space），划分成 512 字节的页。因此，虚拟地址（virtual address）由 23 位的 VPN 和 9 位的偏移量（offset）组成。此外，VPN 的高两位用来区分页属于哪个段（segment）；因此，正如我们之前见过的，这个系统是分页（paging）和分段（segmentation）的混合体。

地址空间（address space）的下半部分称为“进程空间”（process space），每个进程各有一份。进程空间（process space）的前一半称为 P0，里面放着用户程序，还有一个向下增长的堆（heap）；后一半称为 P1，里面是向上增长的栈（stack）。地址空间的上半部分称为系统空间（system space，S），不过只用到了其中的一半。受保护的操作系统代码和数据就放在这里，所有进程也就以这种方式共享了操作系统。

VMS 设计者的一大顾虑是 VAX 硬件的页小得出奇（512 字节）。这个大小是出于历史原因选定的，它有一个根本性的问题：会让简单的线性页表（linear page table）变得过大。因此，VMS 设计者最早确立的目标之一，就是确保 VMS 不会让页表把内存撑爆。

这个系统用两种办法减轻页表给内存带来的压力。第一，VAX-11 把用户地址空间分成两段，每个进程的这两个区域（P0 和 P1）各有一张页表；这样，栈和堆之间未使用的那部分地址空间就不需要页表空间了。基址（base）和界限寄存器（bounds register）的用法和你预想的一样：基址寄存器（base register）保存该段页表的地址，界限寄存器保存页表的大小（即页表项（page table entry）的个数）。

第二，操作系统把用户页表放在内核虚拟内存（kernel virtual memory）中，进一步减轻了内存压力（memory pressure）；用户页表 P0 和 P1 各一张，因此每个进程有两张。这样，分配或扩大页表时，内核从自己位于 S 段的虚拟内存中分配空间。如果内存压力（memory pressure）非常大，内核可以把这些页表的页换出（swap out）到磁盘，从而腾出物理内存（physical memory）另作他用。

补充：通用性的诅咒

操作系统常常受困于所谓的**通用性诅咒**（the curse of generality）：它们要为一大类应用程序和系统提供通用的支持。这个诅咒的根本后果是，操作系统很难对某一种具体环境支持得特别好。对 VMS 来说，这个诅咒实实在在，因为 VAX-11 体系结构有许多种不同的实现。今天也一样：人们指望 Linux 在你的手机、电视机顶盒、笔记本电脑、台式机上都跑得好，也指望它在云数据中心里运行着数千个进程的高端服务器上跑得好。

把页表放在内核虚拟内存（kernel virtual memory）中，意味着地址转换（address translation）会更加复杂。例如，要转换 P0 或 P1 中的一个虚拟地址，硬件首先要在这个进程的 P0 或 P1 页表中查找该页的页表项；而要找到这个页表项，硬件可能得先查位于物理内存中的系统页表；这次转换完成后，硬件才知道页表的那一页在哪里，最后才能得到要访问的内存地址。好在 VAX 有硬件管理的 TLB（hardware-managed TLB），能让这一切快起来：它通常（但愿如此）能绕过这种费力的查找。

真实的地址空间

研究 VMS 有一个很妙的地方：我们可以看到一个真实的地址空间是如何构造的（图 23.1）。到目前为止，我们一直假设地址空间很简单，只有用户代码、用户数据和用户堆（user heap）；但从图中可以看出，真实的地址空间要复杂得多。

例如，代码段（code segment）永远不会从第 0 页开始。第 0 页被标记为不可访问，以便在一定程度上支持检测**空指针**（null-pointer）访问。可见，设计地址空间时要考虑的一点是对调试的支持，这里不可访问的第 0 页就以某种形式提供了这种支持。

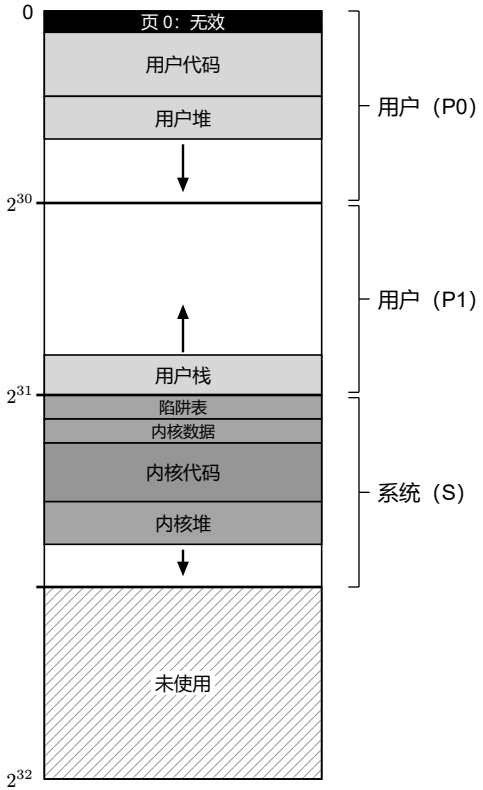


图 23.1：**VAX/VMS 的地址空间**

 © 2008–23, ARPACI-DUSSEAU

补充：为什么空指针访问会导致段错误

现在你应该很清楚，解引用空指针时到底发生了什么。进程像下面这样，产生一个值为 0 的虚拟地址：

```
int *p = NULL; // 设置 p = 0
*p = 10;       // 试图把 10 存到虚拟地址 0
```

硬件试图在 TLB 中查找这个 VPN（这里也是 0），结果 TLB 未命中（TLB miss）。于是去查页表，发现 VPN 0 对应的表项被标记为无效。这样就产生了一次无效访问，控制权转交给操作系统，操作系统很可能会终止这个进程（在 UNIX 系统上，进程会收到一个信号（signal），让它可以对这种错误做出反应；但如果没有捕获这个信号，进程就会被杀掉）。

也许更重要的是，内核虚拟地址空间（kernel virtual address space），也就是内核的数据结构（data structure）和代码，是每个用户地址空间的一部分。上下文切换（context switch）时，操作系统修改 `P0` 和 `P1` 寄存器，让它们指向即将运行的那个进程的页表；但它不修改 `S` 的基址（base）和界限寄存器（bounds register），因此每个用户地址空间里映射的都是“同一套”内核结构。

把内核映射到每个地址空间中，是出于好几方面的考虑。这种做法让内核轻松了不少；例如，当操作系统从用户程序那里拿到一个指针时（比如在 `write()` 系统调用（system call）中），它可以很容易地把数据从这个指针处复制到自己的结构中。操作系统可以按自然的方式编写和编译，不必操心它访问的数据从哪里来。相反，如果内核完全位于物理内存中，要做把页表的页换出到磁盘这类事情就会相当困难；如果给内核单独一个地址空间，在用户应用程序和内核之间移动数据又会变得复杂而痛苦。采用这种做法（如今已被广泛使用）后，在应用程序看来，内核几乎就像一个库，只不过是受保护的库。

关于这个地址空间，最后一点和保护有关。显然，操作系统不希望用户应用程序读写操作系统的数据或代码。因此，硬件必须支持给页设置不同的保护级别。VAX 的做法是在页表的保护位（protection bit）中指明，CPU 必须处于什么特权级别才能访问某一页。这样，系统数据和代码的保护级别就比用户数据和代码更高；用户代码试图访问这类信息时，就会陷入操作系统，然后（你猜对了）这个出错的进程多半会被终止。

页面置换

VAX 中的页表项（PTE）包含以下各位：一个有效位（valid bit）、一个保护字段（4 位）、一个修改位（modified bit，也叫脏位，dirty bit）、一个留给操作系统使用的字段（5 位），最后是物理帧号（PFN），用来记录该页在物理内存中的位置。细心的读者可能已经注意到：没有**访问位**（reference bit）！因此，VMS 的置换算法（replacement algorithm）只能在没有硬件支持的情况下，设法判断哪些页是活跃的。

开发者还担心**内存大户**（memory hogs），也就是那些占用大量内存、让其他程序难以运行的程序。到目前为止我们看过的大多数策略，都难以防止这种霸占行为；例如，LRU 是一种*全局*策略，它不会在进程之间公平地分配内存。

补充：模拟访问位

其实，要想大致知道系统中哪些页正在使用，并不一定需要硬件访问位。早在 20 世纪 80 年代初，Babaoglu 和 Joy 就证明了可以用 VAX 上的保护位（protection bit）来模拟访问位 [BJ81]。基本思路是：如果你想知道系统中哪些页正处于活跃使用状态，就把页表中的所有页都标记为不可访问（但要把进程实际可以访问哪些页的信息保存下来，比如放在页表项的“保留给操作系统的字段”中）。进程访问某一页时，就会陷入操作系统；操作系统随后检查这一页是否确实允许访问，如果是，就把它恢复成正常的保护状态（例如只读或读写）。到了置换的时候，操作系统可以检查哪些页仍然被标记为不可访问，从而知道哪些页最近没被用过。

这种“模拟”访问位的关键，是在降低开销的同时，仍能较好地掌握页的使用情况。操作系统不能太积极地把页标记为不可访问，否则开销会太高；也不能太消极，否则所有页最后都会显示为已访问，操作系统又无从判断该淘汰（evict）哪一页了。

为了解决这两个问题，开发者提出了 **segmented FIFO** 置换策略（replacement policy）[RL81]。思路很简单：每个进程能保留在内存中的页数有一个上限，称为它的**驻留集大小**（resident set size，**RSS**）。这些页都放在一个 FIFO 链表（linked list）里；当进程超出它的 RSS 时，就淘汰“最先进入”的那一页。FIFO 显然不需要任何硬件支持，因此很容易实现。

当然，正如我们前面看到的，纯 FIFO 的表现并不算好。为了改进 FIFO 的性能，VMS 引入了两个**二次机会链表**（second-chance lists）：一个全局的*干净页空闲链表*（clean-page free list）和一个*脏页链表*（dirty-page list），页在被淘汰出内存之前，会先放到这两个链表里。当进程 P 超出它的 RSS 时，就从该进程自己的 FIFO 中移出一页；如果这一页是干净的（没被修改过），就放到干净页链表（clean-page free list）的末尾；如果是脏的（被修改过），就放到脏页链表（dirty-page list）的末尾。

如果另一个进程 Q 需要一个空闲页（free page），它就从全局干净页空闲链表（clean-page free list）中取走第一个空闲页。不过，如果在这一页被回收之前，原来的进程 P 又访问它、发生了缺页，P 就会从空闲链表（free list）或脏页链表（dirty-page list）中把它收回来，从而省掉一次代价高昂的磁盘访问。这两个全局二次机会链表（second-chance list）越大，segmented FIFO 算法的表现就越接近 LRU [RL81]。

VMS 用到的另一项优化，也有助于弥补 VMS 页太小的问题。具体来说，页这么小，交换时的磁盘 I/O 可能非常低效，因为磁盘更擅长大块传输。为了让交换 I/O 更高效，VMS 做了许多优化，其中最重要的是**聚集**（clustering）。有了聚集，VMS 会把全局脏页链表（dirty-page list）中的一大批页归到一起，一次性写到磁盘上（这些页也就变干净了）。大多数现代系统都使用聚集：页可以放在交换空间（swap space）中的任意位置，操作系统因此能把页归成组，用次数更少、规模更大的写操作写出去，从而提高性能。

其他巧妙的技巧

VMS 还有另外两个如今已成标配的技巧：按需置零（demand zeroing）和写时复制（copy-on-write）。我们现在来介绍这些**惰性**（lazy）优化。**按需置零**（demand zeroing）就

是 VMS（以及大多数现代系统）中的一种惰性做法。为了更好地理解它，我们来看往地址空间里添加一页的例子，比如在堆里添加一页。朴素的实现是这样的：操作系统收到往堆里添加一页的请求后，在物理内存中找一页，把它清零（出于安全考虑必须这么做，否则你就能看到别的进程用这一页时留下的内容了！），然后把它映射到你的地址空间中（即按需设置页表，让它指向那个物理页（physical page））。但这种朴素的实现可能代价很高，尤其是进程根本没用到这一页的时候。

有了按需置零（demand zeroing），操作系统在把页加入地址空间时只做很少的工作：在页表中放一个表项，把这一页标记为不可访问。如果进程随后读或写这一页，就会陷入操作系统。处理这个陷阱（trap）时，操作系统会发现这其实是一个按需置零（demand zeroing）的页（通常是靠页表项中“保留给操作系统”那部分里标记的某些位）；这时，操作系统才去做必要的工作：找一个物理页，把它清零，然后映射到进程的地址空间中。如果进程从不访问这一页，这些工作就全都省掉了，这正是按需置零（demand zeroing）的好处。

VMS 中另一项很酷的优化（同样，几乎每个现代操作系统中都有）是**写时复制**（copy-on-write，简称 **COW**）。这个想法至少可以追溯到 TENEX 操作系统 [BB+72]，思路很简单：当操作系统需要把一页从一个地址空间复制到另一个地址空间时，可以不真的复制，而是把它映射到目标地址空间中，并在两个地址空间中都把它标记为只读。如果两个地址空间都只读这一页，那就什么也不用做；操作系统就这样完成了一次快速复制，实际上却没有移动任何数据。

但是，如果其中一个地址空间真的试图写这一页，就会陷入操作系统。操作系统随后会发现这是一个 COW 页，于是（惰性地）分配一个新页，把数据填进去，再把这个新页映射到引发陷阱（trap）的那个进程的地址空间中。然后进程继续运行，这时它已经有了这一页的私有副本。

COW 的用处有很多。比如，任何共享库都可以用写时复制（copy-on-write）的方式映射到许多进程的地址空间中，从而节省宝贵的内存空间。在 UNIX 系统中，由于 `fork()` 和 `exec()` 的语义，COW 就更关键了。你可能还记得，`fork()` 会创建调用者地址空间的一个完全相同的副本；如果地址空间很大，复制它既慢又要搬动大量数据。更糟的是，地址空间的大部分紧接着就会被随后的 `exec()` 调用覆盖：`exec()` 会用即将执行的程序的地址空间替换调用进程的地址空间。如果改用写时复制（copy-on-write）来实现 `fork()`，操作系统就能省掉大部分不必要的复制，既保持了正确的语义，又提高了性能。

23.2 Linux 虚拟内存系统

我们现在来讨论 Linux 虚拟内存系统中一些比较有意思的方面。推动 Linux 发展的，是一群在解决生产环境中真实问题的真正工程师；因此，大量特性被一点点加了进来，最终形成了今天这个功能完备、特性丰富的虚拟内存系统。

我们无法讨论 Linux 虚拟内存的方方面面，但会涉及其中最重要的部分，尤其是它超越 VAX/VMS 这类经典虚拟内存系统的地方。我们也会尽量指出 Linux 和老系统之间的共同之处。

提示：要懒

无论是在生活中还是在操作系统中，懒都可以是一种美德。懒就是把工作推迟到以后再做，这在操作系统中有好几方面的好处。首先，推迟工作可能会降低当前操作的延迟，从而提高响应速度；例如，操作系统常常会立即报告对文件的写操作已经成功，等以后再在后台把它们写到磁盘上。其次，也是更重要的一点，懒有时能让这项工作根本不必做；例如，一次写操作一直推迟到文件被删除，这次写操作就完全省掉了。在生活中懒也有好处：比如，把操作系统课程作业往后拖，你可能会发现作业说明里的错误已经被同学们弄清楚了。不过，课程作业不太可能被取消，所以太懒也会出问题：作业交晚了，分数难看，教授伤心。别让教授伤心！

这里的讨论以 Intel x86 上的 Linux 为主。Linux 虽然能够（而且确实）运行在许多不同的处理器体系结构上，但 x86 上的 Linux 是它最主流、最重要的部署形态，因此也是我们关注的重点。

Linux 的地址空间

和其他现代操作系统一样，也和 VAX/VMS 一样，Linux 的虚拟地址空间[1]由用户部分（存放用户程序代码、栈、堆和其他部分）和内核部分（存放内核代码、栈、堆和其他部分）组成。和那些系统一样，上下文切换时，当前运行的地址空间中，用户部分会随之改变，而内核部分在所有进程中都相同。也和那些系统一样，运行在用户模式（user mode）下的程序不能访问内核的虚拟页（virtual page）；只有陷入内核、切换到特权模式（privileged mode）之后，才能访问这部分内存。

在经典的 32 位 Linux 中，也就是虚拟地址空间为 32 位的 Linux 中，地址空间的用户部分和内核部分以地址 `0xC0000000` 为界，也就是在整个地址空间的四分之三处。因此，虚拟地址 `0` 到 `0xBFFFFFFF` 是用户虚拟地址；其余的虚拟地址（`0xC0000000` 到 `0xFFFFFFFF`）属于内核的虚拟地址空间。64 位 Linux 也有类似的划分，只是分界点略有不同。图 23.2 展示了一个典型的（简化的）地址空间。

Linux 有个小地方挺有意思：它有两种内核虚拟地址（kernel virtual address）。第一种称为**内核逻辑地址**（kernel logical addresses）[O16]。你可以把它看作内核通常意义上的虚拟地址空间；要获得更多这类内存，内核代码只需调用 **`kmalloc`**。大多数内核数据结构都位于这里，比如页表、每个进程的内核栈（kernel stack）等等。和系统中的大多数其他内存不同，内核逻辑内存不能被换出到磁盘。

内核逻辑地址（kernel logical address）最有意思的地方，是它们和物理内存之间的联系。具体来说，内核逻辑地址（kernel logical address）直接映射（direct-mapped）到物理内存的开头部分。因此，内核逻辑地址（kernel logical address）`0xC0000000` 对应物理地址（physical address）`0x00000000`，`0xC0000FFF` 对应 `0x00000FFF`，依此类推。这种直接映射（direct-mapped）意味着两点。第一，内核逻辑地址（kernel logical address）和物理地址之间的相互转换很简单；因此，这些地址常常被直接当作物理地址看待。第二，如

[1]至少在最近因为安全威胁而做出改动之前是这样的。关于这一改动的细节，请阅读下文关于 Linux 安全的小节。

 © 2008–23, ARPACI-DUSSEAU

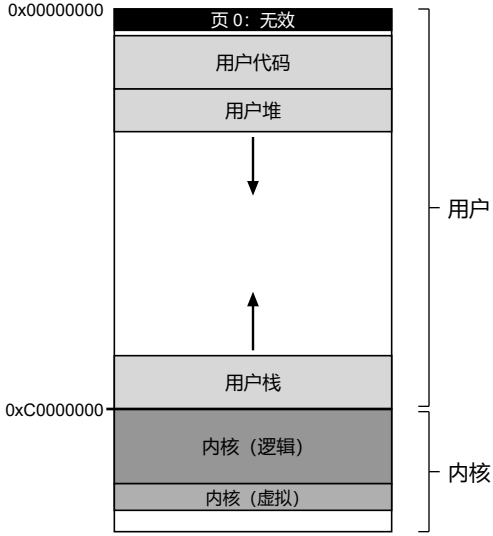


图 23.2：**Linux 的地址空间**

果一块内存在内核逻辑地址（kernel logical address）空间中是连续的，那么它在物理内存中也是连续的。因此，从内核地址空间（kernel address space）这一部分分配的内存，适合用于那些需要连续物理内存才能正确工作的操作，比如通过**直接内存访问**（direct memory access，**DMA**）进出设备的 I/O 传输（本书第三部分会讲到这个话题）。

另一种内核地址是**内核虚拟地址**（kernel virtual address）。要获得这类内存，内核代码要调用另一个分配器（allocator）`vmalloc`，它返回一个指针，指向一块所需大小、在虚拟地址上连续的区域。和内核逻辑内存不同，内核虚拟内存（kernel virtual memory）通常不连续：每个内核虚拟页都可能映射到不相邻的物理页（因此不适合用于 DMA）。不过也正因为如此，这类内存更容易分配，所以常用于大缓冲区（buffer），因为在这种场合，要找到一大块连续的物理内存会很困难。

在 32 位 Linux 中，内核虚拟地址（kernel virtual address）存在的另一个原因是，它们让内核能够寻址超过（大约）1 GB 的内存。多年以前，机器的内存比这少得多，能否访问超过 1 GB 的内存根本不是问题。但技术不断进步，很快内核就需要用上更多内存了。内核虚拟地址（kernel virtual address）不必和物理内存严格一一对应，这才让这一点成为可能。不过，转向 64 位 Linux 之后，这种需求就没那么迫切了，因为内核不再局限于虚拟地址空间中最后那 1 GB。

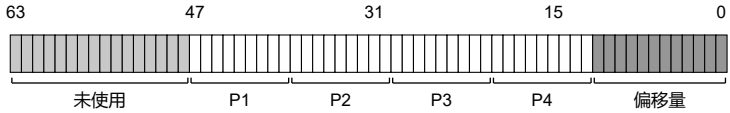
页表结构

我们关注的是 x86 上的 Linux，所以下面围绕 x86 提供的页表结构来讨论，它决定了 Linux 能做什么、不能做什么。如前所述，x86 提供了一种由硬件管理的多级页表（multi-level page table）结构，每个进程一张页表；操作系统只需在自己的内存中建立映射，把一个特权寄存器指向页目录（page directory）的起始位置，剩下的事情就交给硬件处理。不出所

料，操作系统会在创建进程、删除进程以及上下文切换时介入，确保每种情况下硬件 MMU 用来转换的都是正确的页表。

近年来最大的变化，可能就是前面简单提到的从 32 位 x86 转向 64 位 x86。正如在 VAX/VMS 系统中看到的，32 位地址空间已经存在了很长时间，随着技术的发展，它终于开始真正成为程序的限制。虚拟内存让系统编程（systems programming）变得容易，但现代系统动辄有好几 GB 内存，32 位已经不够用来引用其中的每一个字节了。因此，下一次飞跃就成了必然。

转向 64 位地址，对 x86 页表结构的影响和预想的一样。x86 用的是多级页表，当前的 64 位系统使用四级页表。不过，64 位虚拟地址空间目前还没有全部用上，只用了低 48 位。因此，虚拟地址可以看成下面这样：



从图中可以看到，虚拟地址的高 16 位没有使用（因此在转换中不起任何作用），页大小（page size）是 4 KB，所以低 12 位用作偏移量，直接使用，不参与转换；剩下中间的 36 位虚拟地址参与转换。地址中的 P1 部分用来索引最顶层的页目录，然后从这里开始一级一级地转换，直到用 P4 去索引实际的那一页页表，得到所需的页表项。

随着系统内存进一步增大，这个庞大地址空间中会有更多部分被启用，从而出现五级、最终甚至六级的页表树结构。想象一下：只是为了弄清某块数据在内存中的位置，一次简单的页表查找就要经过六级转换。

大页支持

Intel x86 允许使用多种页大小，不只是标准的 4 KB 页。具体来说，近来的设计在硬件上支持 2 MB 甚至 1 GB 的页。因此，随着时间推移，Linux 也逐步支持应用程序使用这些**大页**（Linux 里叫 huge pages）。

前面暗示过，使用大页（huge page）有很多好处。正如在 VAX/VMS 中看到的，大页（huge page）可以减少页表中所需的映射数量；页越大，映射就越少。不过，页表项变少并不是推动大页（huge page）的主要动力，真正的动力是更好的 TLB 表现以及随之而来的性能提升。

进程频繁使用大量内存时，它的地址转换很快就会把 TLB 装满。如果这些转换针对的都是 4 KB 的页，那么不引发 TLB 未命中就能访问的内存总量就很少。结果，对于运行在好几 GB 内存机器上的现代“大内存”工作负载（workload）来说，这会带来明显的性能损失；最近的研究表明，有些应用程序要花 10% 的周期来处理 TLB 未命中 [B+13]。

大页（huge page）占用的 TLB 槽位更少，进程因此能在不发生 TLB 未命中的情况下访问一大片内存，这是大页的主要优点。不过，大页（huge page）还有其他好处：TLB 未命中的处理路径更短，也就是说，真的发生 TLB 未命中时，处理得更快。此外，（在某些场景下）分配也可以非常快，这个好处虽小，有时却很重要。

提示：考虑渐进式改进

人生中，你常常会被鼓励去做一个革命者。“要胸怀大志!”他们说。“去改变世界!”他们喊。你也明白这为什么诱人：有些时候确实需要大变革，大力推动它也很有道理。而且，照这么做的话，至少他们可能就不会再冲你嚷嚷了。

然而在很多情况下，更慢、更渐进的方法才是正确的做法。本章中 Linux 的大页（huge page）就是工程上渐进主义的一个例子：开发者没有摆出原教旨主义者的姿态、坚称大页就是未来，而是采取了稳妥的做法：先为它引入专门的支持，进一步了解它的优缺点，等到真有必要时，才为所有应用程序加入更通用的支持。

渐进主义有时会遭人轻视，但它常常能带来缓慢、深思熟虑而又合理的进步。构建系统时，这样的方法也许正是你需要的。其实，人生或许也是如此。

Linux 对大页（huge page）的支持有个有意思的地方：它是一步一步加进来的。起初，Linux 开发者知道这种支持只对少数应用程序重要，比如对性能要求很高的大型数据库。因此，他们决定让应用程序显式申请用大页（huge page）来分配内存（通过 `mmap()` 或 `shmget()` 调用）。这样，大多数应用程序不受影响（继续只使用 4 KB 的页）；少数要求很高的应用程序需要修改代码来使用这些接口，但对它们来说，这点麻烦是值得的。

最近，越来越多的应用程序都需要更好的 TLB 表现，于是 Linux 开发者加入了**透明**大页（transparent huge page）支持。启用这一特性后，操作系统会自动寻找机会分配大页（huge page，通常是 2 MB，在某些系统上是 1 GB），应用程序无须修改。

大页（huge page）也不是没有代价。最大的潜在代价是**内部碎片**（internal fragmentation），也就是一个很大的页却只用了很少一部分。这种浪费可能让内存被又大又没怎么用的页占满。交换（如果启用的话）和大页（huge page）也配合得不好，有时会让系统的 I/O 量大大增加。在另一些情况下，分配的开销也可能很大。总的来说，有一点很清楚：多年来为系统立下汗马功劳的 4 KB 页大小，已经不再是以前那种万能的解决方案了；内存容量不断增长，要求我们把大页（huge page）和其他方案纳入考虑，作为虚拟内存系统必要演进的一部分。Linux 慢慢采纳这项基于硬件的技术，本身就说明变化即将到来。

页缓存

为了降低访问持久存储（persistent storage，本书第三部分的重点）的开销，大多数系统都使用积极的**缓存**（caching）子系统，把常用的数据保留在内存中。在这方面，Linux 和传统操作系统没什么不同。

Linux 的**页缓存**（page cache）是统一的，它把来自三个主要来源的页保留在内存中：**内存映射文件**（memory-mapped files）；来自设备的文件数据和元数据（metadata），它们通常是通过对文件系统（file system）发出 `read()` 和 `write()` 调用来访问的；以及构成每个进程的堆页（heap page）和栈页（stack page），这类页有时称为**匿名内存**（anonymous memory），因为它背后没有具名文件，只有交换空间。这些页保存在一张**页缓存散列表**（page cache hash table）中，需要时可以快速查到。

页缓存记录每个条目是**干净的**（clean，读过但未更新）还是**脏的**（dirty，也叫**已修改**，modified）。脏数据会由名为 `pdflush` 的后台线程（thread）定期写回后备存储（backing store）：文件数据写回对应的文件，匿名区域写到交换空间；这样就能确保修改过的数据最终会被写回持久存储。这种后台活动要么在经过一段时间后进行，要么在脏页（dirty page）太多时进行（两者都是可配置的参数）。

有时系统内存不足，Linux 就必须决定把哪些页踢出内存以腾出空间。为此，Linux 使用了一种改进版的 **2Q** 置换算法（replacement algorithm）[JS94]，下面介绍一下。

基本思路很简单：标准的 LRU 置换很有效，但某些常见的访问模式会让它失灵。例如，如果一个进程反复访问一个大文件（尤其是大小接近内存甚至更大的文件），LRU 会把其他所有文件都踢出内存。更糟的是：把这个文件的部分内容保留在内存中并没有用，因为它们在被踢出内存之前根本不会被再次访问。

Linux 版的 2Q 置换算法维护两个链表，并把内存分给这两个链表，以此解决这个问题。一页第一次被访问时，会被放到一个队列（queue）中（原论文中称为 A1，在 Linux 中称为**不活跃链表**，inactive list）；当它再次被访问时，就被提升到另一个队列中（原论文中称为 Aq，在 Linux 中称为**活跃链表**，active list）。需要置换时，从不活跃链表（inactive list）中挑选候选页。Linux 还会定期把活跃链表（active list）底部的页移到不活跃链表中，让活跃链表保持在页缓存总大小的三分之二左右 [G04]。

理想情况下，Linux 会按严格的 LRU 顺序管理这些链表，但前几章讨论过，这样做代价很高。因此，和许多操作系统一样，Linux 用的是一种近似 LRU（类似于**时钟**置换）。

这种 2Q 方法的表现大体上和 LRU 相当接近，但它专门处理了循环访问大文件的情况：把这类循环访问的页限制在不活跃链表（inactive list）里。由于这些页在被踢出内存之前根本不会被再次访问，它们就不会把活跃链表（active list）中其他有用的页挤出去。

安全与缓冲区溢出

现代虚拟内存系统（Linux、Solaris 或某个 BSD 变种）和古老的系统（VAX/VMS）之间最大的区别，可能就在于现代对安全的重视。保护一直是操作系统重点关注的问题，而如今机器之间的互联程度前所未有，开发者实现了各种防御措施来阻止那些狡猾的黑客夺取系统控制权，也就不足为奇了。

一大威胁来自**缓冲区溢出**（buffer overflow）攻击[2]，它可以用来攻击普通的用户程序，甚至内核本身。这类攻击的思路是在目标系统中找到一个 bug，让攻击者能够向目标的地址空间注入任意数据。这类漏洞的成因有时是这样的：开发者（错误地）假定输入不会太长，于是（轻信地）把输入复制到缓冲区里；而输入实际上太长，就会溢出缓冲区，覆盖目标的内存。像下面这样看起来人畜无害的代码，就可能是问题的根源：

```
int some_function(char *input) {
    char dest_buffer[100];
    strcpy(dest_buffer, input); // 糟糕，复制时没有检查长度！
}
```

[2]关于这个话题的一些细节和链接，请参见 https://en.wikipedia.org/wiki/Buffer_overflow，其中还提到了安全黑客 Elias Levy（又名“Aleph One”）的那篇著名文章。

补充：无处不在的内存映射

内存映射（memory mapping）比 Linux 早出现好些年，Linux 和其他现代系统中有很多地方都用到了它。思路很简单：对一个已经打开的文件描述符（file descriptor）调用 `mmap()`，进程就会得到一个指针，指向一块虚拟内存区域的开头，文件的内容看起来就位于这块区域中。之后，进程只需用这个指针做简单的解引用，就能访问文件的任何部分。

访问内存映射文件（memory-mapped file）中尚未调入内存的部分时，会触发**缺页异常**（page fault，也叫缺页中断），这时操作系统会把相关数据调入内存，并相应地更新进程的页表，使这些数据可以访问（即**按需分页**，demand paging）。

每个普通的 Linux 进程都会用到内存映射文件（memory-mapped file），即使 `main()` 中的代码并没有直接调用 `mmap()`，这是 Linux 把可执行文件和共享库的代码加载进内存的方式决定的。下面是 `pmap` 命令行工具的输出（经过大幅删减），它显示了一个正在运行的程序（本例中是 shell，即 `tcsh`）的虚拟地址空间由哪些不同的映射组成。输出有四列：映射的虚拟地址、大小、该区域的保护位（protection bit），以及映射的来源：

```
0000000000400000   372K r-x-- tcsh
00000000019d5000  1780K rw---   [anon ]
00007f4e7cf06000  1792K r-x-- libc-2.23.so
00007f4e7d2d0000    36K r-x-- libcrypt-2.23.so
00007f4e7d508000   148K r-x-- libtinfo.so.5.9
00007f4e7d731000   152K r-x-- ld-2.23.so
00007f4e7d932000    16K rw---   [stack ]
```

从输出可以看出，`tcsh` 二进制文件的代码，以及来自 `libc`、`libcrypt`、`libtinfo` 的代码和动态链接器（dynamic linker）本身（`ld.so`）的代码，都被映射到了地址空间中。此外还有两个匿名区域：堆（第二项，标为 `anon`）和栈（标为 `stack`）。内存映射文件（memory-mapped file）为操作系统构建现代地址空间提供了一种直接而高效的方法。

很多情况下，这样的溢出并不是灾难性的：比如，无意中传给用户程序甚至操作系统的错误输入，多半只会让它崩溃，仅此而已。然而，恶意的程序员可以精心构造溢出缓冲区的输入，把自己的代码注入目标系统，从而基本上接管这个系统，让它听从自己的指挥。如果攻击联网的用户程序得手，攻击者就可以执行任意计算，甚至把被攻陷系统的计算周期租出去；如果攻击操作系统本身得手，就能访问更多资源，这属于所谓的**权限提升**（privilege escalation，即用户代码获得内核访问权限）。如果你还没猜到的话：这些都是坏事。

防御缓冲区溢出的第一种、也是最简单的方法，是禁止执行位于地址空间某些区域（例如栈）中的任何代码。AMD 在它的 x86 版本中引入的 **NX 位**（NX bit，No-eXecute 的缩写，即禁止执行；Intel 的处理器上现在也有类似的 XD 位）就是这样一种防御：如果某页对应的页表项中设置了这一位，就禁止执行该页上的任何代码。这样，攻击者注入目标栈中的代码就无法执行，问题也就得到了缓解。

 © 2008–23, ARPACI-DUSSEAU

然而，聪明的攻击者毕竟……很聪明：即使攻击者无法直接注入代码，恶意代码照样可以执行任意的代码序列。这种思想最一般的形式称为**返回导向编程**（return-oriented programming，**ROP**）[S07]，它确实相当高明。ROP 背后的观察是：任何程序的地址空间中都有大量零碎的代码片段（用 ROP 的术语来说叫 **gadget**），链接了庞大 C 库的 C 程序尤其如此。因此，攻击者可以覆盖栈，让当前执行函数的返回地址指向所需的某条恶意指令（或一串指令），这些指令后面紧跟着一条返回指令。把大量 gadget 串在一起（即确保每次返回都跳到下一个 gadget），攻击者就能执行任意代码。太神奇了！

为了防御 ROP（包括它更早的形式，即 **return-to-libc 攻击** [S+04]），Linux（以及其他系统）又加入了一种防御，称为**地址空间布局随机化**（address space layout randomization，**ASLR**）。操作系统不再把代码、栈和堆放在虚拟地址空间中的固定位置，而是随机安排它们的位置，这样一来，要构造实施这类攻击所需的复杂代码序列就相当困难了。因此，针对有漏洞的用户程序的攻击，大多只会让程序崩溃，而无法夺取正在运行的程序的控制权。

有意思的是，你很容易就能亲眼看到这种随机性（randomness）。下面这段代码可以在现代 Linux 系统上演示：

```
int main(int argc, char *argv[]) {
    int stack = 0;
    printf("%p\n", &stack);
    return 0;
}
```

这段代码只是打印出栈上一个变量的（虚拟）地址。在较老的、没有 ASLR 的系统中，每次打印出的值都相同。但从下面可以看到，每次运行这个值都会变：

```
prompt> ./random
0x7ffd3e55d2b4
prompt> ./random
0x7ffe1033b8f4
prompt> ./random
0x7ffe45522e94
```

ASLR 对用户级程序是一种非常有用的防御，所以它也被引入了内核，这个特性的名字毫无新意，就叫**内核地址空间布局随机化**（kernel address space layout randomization，**KASLR**）。不过事实表明，内核可能还有更大的问题要应对，下面就来讨论。

其他安全问题：Meltdown 和 Spectre

在我们写下这段文字时（2018 年 8 月），系统安全领域已经被两种新的、相互关联的攻击搅得天翻地覆。第一种叫 **Meltdown**（熔断），第二种叫 **Spectre**（幽灵）。它们几乎同时被四组不同的研究人员和工程师发现，让人们开始深刻质疑计算机硬件及其上的操作系统所提供的基本保护。详细描述这两种攻击的论文见 `spectreattack.com`；两者之中，Spectre 被认为更麻烦。

这两种攻击利用的是同一个弱点：现代系统中的 CPU 为了提高性能，会在幕后施展各种疯狂的技巧。处于问题核心的一类技术叫作**推测执行**（speculative execution）：CPU 猜测接下来很快会执行哪些指令，并提前开始执行。如果猜对了，程序就跑得更快；如果猜错

了，CPU 会撤销这些指令对体系结构状态（例如寄存器）的影响，然后再试一次，这次走正确的路径。

推测执行（speculative execution）的问题在于，它往往会在系统的各个部分留下执行的痕迹，比如处理器缓存、分支预测器等。问题就来了：正如这些攻击的作者所展示的，这些状态可能让内存中的内容暴露在攻击之下，甚至包括我们以为受 MMU 保护的内存。

因此，加强内核保护的一条途径，是从每个用户进程中移除尽可能多的内核地址空间，改为给大多数内核数据单独准备一张内核页表（这称为**内核页表隔离**，kernel page-table isolation，**KPTI**）[G+17]。这样一来，内核的代码和数据结构不再映射到每个进程中，进程里只保留最起码的一小部分；相应地，切换到内核时，还得切换到内核页表（kernel page table）。这样做提高了安全性，堵住了一些攻击途径，但也有代价：性能。切换页表的代价很高。唉，安全的代价：便利和性能。

遗憾的是，KPTI 并不能解决上面列出的所有安全问题，只能解决其中一部分。而像关闭推测执行（speculative execution）这样简单的解决办法也没什么意义，因为那样系统会慢上几千倍。因此，如果你对系统安全感兴趣，现在可真是个有意思的时代。

要真正理解这些攻击，你（很可能）得先学习很多东西。先从理解现代计算机体系结构入手（可以看这方面的高级书籍），重点关注推测执行（speculative execution）以及实现它所需的各种机制。一定要读一读上面提到的网站上关于 Meltdown 和 Spectre 攻击的内容；那里其实也有一份关于推测执行（speculative execution）的实用入门介绍，所以也许是个不错的起点。还要研究操作系统，寻找更多的漏洞。谁知道还有哪些问题没被发现呢？

23.3 小结

现在，你已经从头到尾完整地审视了两个虚拟内存系统。但愿大部分细节都很容易理解，因为你应该已经很好地掌握了基本的机制和策略。关于 VAX/VMS 的更多细节，可以参阅 Levy 和 Lipman 那篇出色（而且很短）的论文 [LL82]。我们鼓励你读一读，通过它，你可以看到这几章内容的原始出处是什么样子。

你还对 Linux 有了一些了解。Linux 虽然是一个庞大而复杂的系统，但它继承了过去的许多好思想，其中很多我们没有篇幅详细讨论。例如，Linux 在 `fork()` 时对页进行惰性的写时复制（copy-on-write），避免不必要的复制，从而降低开销。Linux 也通过内存映射 `/dev/zero` 设备对页做按需置零（demand zeroing），还有一个后台换页守护进程（swap daemon）**`swapd`**，负责把页换出到磁盘以减轻内存压力（memory pressure）。事实上，Linux 的虚拟内存充满了从过去借鉴来的好思想，同时也包含了许多它自己的创新。

想了解更多，可以看看这几本还不错（但可惜已经过时）的书 [BC05, G04]。我们鼓励你自己去读一读，因为我们只能从复杂性的海洋中取出最微小的一滴。不过，万事总得有个开头。海洋不就是无数水滴汇成的吗？[M04]

 © 2008–23, ARPACI-DUSSEAU

参考文献

[B+13] "Efficient Virtual Memory for Big Memory Servers" by A. Basu, J. Gandhi, J. Chang, M. D. Hill, M. M. Swift. ISCA '13, June 2013, Tel-Aviv, Israel. *A recent work showing that TLBs matter, consuming 10% of cycles for large-memory workloads. The solution: one massive segment to hold large data sets. We go backward, so that we can go forward!*

[BB+72] "TENEX, A Paged Time Sharing System for the PDP-10" by D. G. Bobrow, J. D. Burchfiel, D. L. Murphy, R. S. Tomlinson. CACM, Volume 15, March 1972. *An early time-sharing OS where a number of good ideas came from. Copy-on-write was just one of those; also an inspiration for other aspects of modern systems, including process management, virtual memory, and file systems.*

[BJ81] "Converting a Swap-Based System to do Paging in an Architecture Lacking Page-Reference Bits" by O. Babaoglu, W. N. Joy. SOSP '81, Pacific Grove, California, December 1981. *How to exploit existing protection machinery to emulate reference bits, from a group at Berkeley working on their own version of* UNIX*: the* **Berkeley Systems Distribution** *(***BSD***). The group was influential in the development of virtual memory, file systems, and networking.*

[BC05] "Understanding the Linux Kernel" by D. P. Bovet, M. Cesati. O'Reilly Media, November 2005. *One of the many books you can find on Linux, which are out of date, but still worthwhile.*

[C03] "The Innovator's Dilemma" by Clayton M. Christenson. Harper Paperbacks, January 2003. *A fantastic book about the disk-drive industry and how new innovations disrupt existing ones. A good read for business majors and computer scientists alike. Provides insight on how large and successful companies completely fail.*

[C93] "Inside Windows NT" by H. Custer, D. Solomon. Microsoft Press, 1993. *The book about Windows NT that explains the system top to bottom, in more detail than you might like. But seriously, a pretty good book.*

[G04] "Understanding the Linux Virtual Memory Manager" by M. Gorman. Prentice Hall, 2004. *An in-depth look at Linux VM, but alas a little out of date.*

[G+17] "KASLR is Dead: Long Live KASLR" by D. Gruss, M. Lipp, M. Schwarz, R. Fellner, C. Maurice, S. Mangard. Engineering Secure Software and Systems, 2017. Available: https://gruss.cc/files/kaiser.pdf *Excellent info on KASLR, KPTI, and beyond.*

[JS94] "2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm" by T. Johnson, D. Shasha. VLDB '94, Santiago, Chile. *A simple but effective approach to building page replacement.*

[LL82] "Virtual Memory Management in the VAX/VMS Operating System" by H. Levy, P. Lipman. IEEE Computer, Volume 15:3, March 1982. *Read the original source of most of this material. Particularly important if you wish to go to graduate school, where all you do is read papers, work, read some more papers, work more, eventually write a paper, and then work some more.*

[M04] "Cloud Atlas" by D. Mitchell. Random House, 2004. *It's hard to pick a favorite book. There are too many! Each is great in its own unique way. But it'd be hard for these authors not to pick "Cloud Atlas", a fantastic, sprawling epic about the human condition, from where the the last quote of this chapter is lifted. If you are smart — and we think you are — you should stop reading obscure commentary in the references and instead read "Cloud Atlas"; you'll thank us later.*

[O16] "Virtual Memory and Linux" by A. Ott. Embedded Linux Conference, April 2016. https://events.static.linuxfound.org/sites/events/files/slides/elc_2016_mem.pdf . *A useful set of slides which gives an overview of the Linux VM.*

[RL81] "Segmented FIFO Page Replacement" by R. Turner, H. Levy. SIGMETRICS '81, Las Vegas, Nevada, September 1981. *A short paper that shows for some workloads, segmented FIFO can approach the performance of LRU.*

[S07] "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)" by H. Shacham. CCS '07, October 2007. *A generalization of return-to-libc. Dr. Beth Garner said in Basic Instinct, "She's crazy! She's brilliant!" We might say the same about ROP.*

[S+04] "On the Effectiveness of Address-space Randomization" by H. Shacham, M. Page, B. Pfaff, E. J. Goh, N. Modadugu, D. Boneh. CCS '04, October 2004. *A description of the return-to-libc attack and its limits. Start reading, but be wary: the rabbit hole of systems security is deep…*

 © 2008–23, ARPACI-DUSSEAU